ClauseGuard: Legal Contract Complexity and Predatory Clause Detector

CH.PRANATHI
Roll No. 23K91A6744
Dept. of CSE (Data Science)
[COLLEGE NAME]
[CITY]
[EMAIL]

G.PRANAVI
Roll No. 23K91A6762
Dept. of CSE (Data Science)
[COLLEGE NAME]
[CITY]
[EMAIL]

G.CHARAN JOSEPH
Roll No. 23K91A6753
Dept. of CSE (Data Science)
[COLLEGE NAME]
[CITY]
[EMAIL]

G.BHAGWATH
Roll No. 23K91A6760
Dept. of CSE (Data Science)
[COLLEGE NAME]
[CITY]
[EMAIL]

[GUIDE NAME]
[DESIGNATION], Project Guide
Dept. of CSE (Data Science)
[COLLEGE NAME]
[CITY]
[EMAIL]

***Abstract*—**Freelancers, individuals and small businesses often sign contracts they cannot assess, and professional review is costly. This paper presents ClauseGuard, a web system that reads a PDF or DOCX contract, splits it into clauses, labels each clause with one of 20 CUAD categories using Gemini embeddings and a nearest-centroid rule, flags eight predatory patterns tied to Indian contract law with regular expressions, detects three risky clause combinations in a graph, grades textual complexity from A to F and asks a language model for plain-language reasons and safer wording. Classification and predatory detection are evaluated separately. On 240 held-out CUAD spans, top-1 accuracy is 0.792 and top-3 accuracy 0.979. The rules score precision 1.00 and recall 0.94 on the 40 clauses written with them, but precision 0.79 and recall 0.46 on an independent probe set. PDF line breaks hide two of eight rules on identical text, and two classifier thresholds never trigger. Sixty-eight automated tests pass. The limits are reported plainly.

***Index Terms*—**clause classification, complexity scoring, contract analysis, Indian contract law, legal NLP, predatory clause detection, text embeddings.

I. INTRODUCTION

Individuals, freelancers, gig workers and small businesses now sign digital contracts without a lawyer. Such contracts can hold unfair termination rights, penalties that bear no relation to loss, unlimited liability, one-sided arbitration and automatic renewals, and the weaker party seldom notices them. Professional review is slow and expensive, so the information gap stays. Large language models can read a contract and explain it in plain words, but a general chatbot is not grounded in the contract's clauses and gives no clause-level reference. Commercial analysis tools and research systems mostly target US or EU law, classify one clause at a time, and often mix up two different questions: what type a clause is, and whether it is predatory.

The base paper [1] builds a retrieval-augmented legal assistant and contract checker for Saudi law. We adopt its jurisdiction-aware, language-model-explained design for Indian contracts, but our build differs from the original plan in three ways: Gemini replaces the Claude API, vectors are kept in SQLite and compared with NumPy instead of pgvector, and a per-contract NetworkX graph replaces Neo4j. This paper therefore reports what was built and measured, not what was planned. Its contributions are:

1) an end-to-end pipeline for PDF and DOCX contracts: segmentation, 20-category CUAD classification, eight rules tagged with Indian statutes, three risky combinations, an A-F complexity grade, and LLM-written reasons, safer wording and summaries;
2) a separate evaluation of classification (240 CUAD spans) and predatory detection (a 40-clause set, an independent 40-clause probe set, a CUAD cross-check and a 510-contract run);
3) measured findings about the design: where the rules and the classifier fail, what PDF input does to detection, and which thresholds never act;
4) a tested React and FastAPI application with 68 passing tests and real captured output screens.

Section II reviews related work, Section III describes the method, Section IV reports results and Section V concludes.

A. System Hardware

All measurements were taken on one desktop with a 12th-generation Intel Core i5-12500 (12 threads), 15.7 GB of memory and no GPU in use, running Windows 11. Embeddings and text generation are remote Gemini API calls, so no model was trained or run locally.

B. Software Requirements

The back end uses Python 3.11.9, FastAPI 0.115, SQLAlchemy 2 over SQLite, PyMuPDF and python-docx for parsing, NumPy, NetworkX 3.3 [2], the google-generativeai client, JSON Web Tokens [3] and bcrypt [4]. The front end is React 19 with Vite and Tailwind on Node 24.19.0. Baselines use scikit-learn [5]; tests use pytest 9.1.1.

II. LITERATURE SURVEY

Base paper. Almutairy et al. [1] present SmartLaw Advisor, a Saudi-law assistant with two functions: a legal chatbot and a contract analyser. The chatbot uses Gemini 1.5 Flash with a standard retrieval-augmented generation (RAG) pipeline [6] over a custom Arabic corpus of Saudi regulations: semantic chunking, GATE-AraBERT-v1 embeddings in a Chroma store (a FAISS index of about 18,400 vectors is also described), cross-encoder re-ranking, OCR-driven normalisation and JSON-constrained clause extraction at temperature 0.2. The contract analyser checks each contract against a checklist of essential clauses validated by Saudi practitioners and applies an LLM-assisted semantic

check. The system was evaluated on 150 legal questions in ten domains and on 50 real contract drafts. Gemini 1.5 Flash answered 148 of 150 questions correctly (98.67%) in 1.63 s on average, ahead of GPT-3.5 (98.00%, 147), GPT-4 (97.33%, 146) and ALLaM (96.00%, 144). Contract-analysis results are qualitative: the authors inspect the clauses returned by each model. Reported failures are retrieval across chunk boundaries and clause misclassification. The work is limited to Arabic and Saudi law, uses a small expert-reviewed dataset, and has neither graph reasoning, clause-combination analysis nor a complexity score.

Related work. ContractEval [7] benchmarks 19 LLMs (four proprietary, 15 open-source) on the CUAD test data [8] over 41 risk categories, using F1, F2, Jaccard overlap and the rate of wrongly answering "no related clause"; proprietary models lead and the top two reach F2 of 0.672 and 0.678. Lincoln et al. [9] compare a domain-trained small language model with five frontier models on 24 held-out contracts (508 field instances, 26 targets) and report a macro F1 of 0.812 and a micro F1 of 0.842, with extraction cost lower by 78% to 97%. The legal document analyser of [10] extracts ten clause types with cloud services and a language model and scores missing or risky clauses against a template on a 0-100 scale; it reports 94.2% mean extraction accuracy on 20 synthetic contracts and a Pearson correlation of 0.87 between its score and expert ratings on 30 contracts. CUAD [8] supplies 510 expert-annotated contracts with 41 categories. LEGAL-BERT [11] adapts BERT [12] to legal text by pre-training on legal corpora, and Sentence-BERT [13] and Gemini Embedding [14] provide the dense sentence vectors on which nearest-centroid and retrieval methods rest. TF-IDF weighting [15] is the classical sparse baseline, and the Flesch score [16] the classical readability measure.

TABLE I
RELATED WORK

Work	Approach	Data	Reported result	Limitation
[1]	Gemini 1.5 Flash, RAG, clause checklist	Saudi corpus; 150 questions; 50 contracts	98.67% (148/150) on questions	Saudi law; contract part qualitative
[7]	19 LLMs, clause-level risk	CUAD test data, 41 categories	Top two F2 0.672, 0.678	Extraction; US contracts
[9]	Small model vs 5 frontier LLMs	24 contracts, 508 fields	Macro F1 0.812; micro 0.842	Field extraction, not risk
[10]	Cloud NLP, template risk score	20 synthetic, 30 sample contracts	94.2% accuracy; r = 0.87	Synthetic; missing clauses only
[8]	Annotated corpus	510 contracts, 41 categories	Dataset	US SEC filings
[11]	Legal-domain BERT	Legal corpora	No result quoted	No contract-risk output

Reading across the work. Three points shape our design. First, the base paper [1] shows that a jurisdiction-specific language-model assistant can reach high answer accuracy, but its contract part is judged by reading outputs, with no count of missed or false flags; we therefore score flags against labelled clauses. Second, the CUAD-based studies [7], [8] treat "which clause type is this" and "find the clause that answers a risk question" as extraction tasks over US filings; none says whether a clause is unfair under a given law, so a predatory label cannot be read off a CUAD category and must come from rules or from new annotation. Third, the template-based scoring of [10] reports a strong correlation with expert ratings, but on synthetic and sample contracts and for missing clauses, not for one-sided wording; we keep clause-level evidence and test the rules on phrasing they were not written for.

Gap. The tools and studies above concentrate on US, EU or Saudi law, treat clauses singly, and score extraction or question answering. We found none that combines Indian-law predatory rules, risky clause combinations and a complexity grade, or that reports clause classification and predatory detection as two separate measurements.

III. METHODOLOGY

A. Architecture and Main Flow

ClauseGuard is a React app and a FastAPI back end with three routers (authentication, contracts, evaluation), nine endpoints and four SQLite tables (Fig. 1). A signed-in user uploads a file; the pipeline of Fig. 2 parses, segments, classifies, flags, combines, scores and summarises it and stores the result. Only embeddings, tie-breaks, reasons, safer wording and summaries call Gemini; the decision whether a clause is risky is taken by local rules.

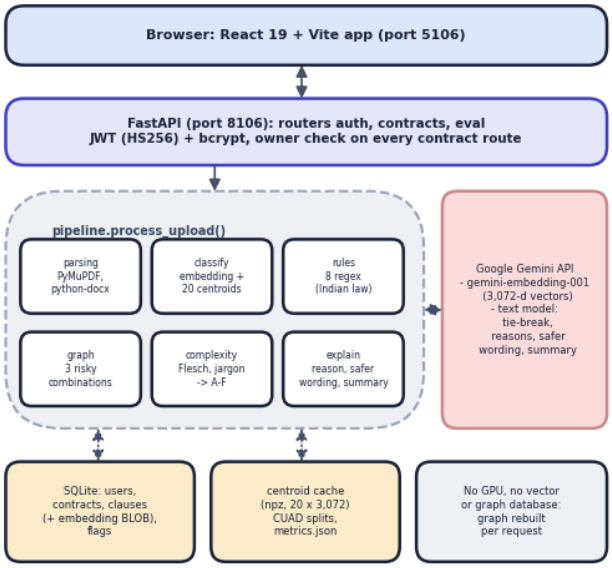


Fig. 1. System architecture of ClauseGuard.

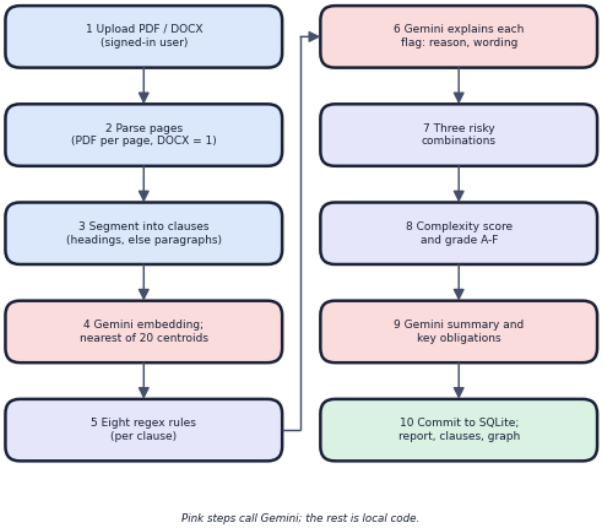


Fig. 2. Main flow from upload to report.

B. Data

Clause types come from CUAD [8] (510 contracts, CC BY 4.0). We use 20 substantive categories: 500 training spans (25 per category) build the centroids and 240 held-out spans (12 per category) test them; the split is seeded but made by span text, not by contract, so 192 of 240 test spans share a contract with a training span. Predatory detection uses the 40 labelled clauses shipped with the project, which were written by the author of the rules, and a probe set written for this study: 24 predatory paraphrases (three per rule, mostly without the rule's keywords) and 16 benign look-alikes. Three sample contracts drive the screens: an invented one-sided freelance agreement, an invented fair consulting agreement and a real CUAD supply agreement.

Why these choices. A nearest-centroid rule needs no training run, can be rebuilt in seconds from 500 labelled spans, and is easy to audit: the label of a clause is the category whose average example it most resembles. Training spans are embedded as documents and test clauses as queries, as the Gemini embedding interface requires, and each clause vector is stored as a float32 BLOB beside the clause. Deterministic rules were preferred for the risk decision because they are reproducible and inspectable: the same clause always gets the same flag, and the language model only explains a flag that already exists. The cost is that a rule sees words, not meaning, which Section IV measures.

C. Segmentation and Classification

A PDF is read page by page, a DOCX as one page. A page is split at numbered headings ("1.", "Section 2.1"); with fewer than two headings it is split into blank-line paragraphs merged to about 400 characters, and fragments under 25 characters are dropped. Each clause is embedded with gemini-embedding-001 (3,072 dimensions). With the centroid of category k taken as the mean vector of its 25 training spans,

$$s_k = \cos(e, \text{centroid}_k), \quad k = 1..20 \tag{1}$$

the label is the category with the highest s. If the gap between the best two is below 0.03 and the best is below 0.60, the language model chooses among the top three:

$$\text{tie-break if } (s_1 - s_2 < 0.03) \wedge (s_1 < 0.60) \tag{2}$$

and a clause whose final confidence is below 0.35 is labelled "Other / Unclassified".

D. Rules, Combinations and Complexity

Eight case-insensitive regular expressions detect restraint of trade (Indian Contract Act s. 27 [17]), penalty against liquidated damages (s. 74), unilateral termination, unlimited liability, one-sided arbitration (Arbitration and Conciliation Act [18]), automatic renewal and no-refund terms (Consumer Protection Act [19]) and intellectual-property overreach (Copyright Act [20]). Each hit becomes one flag; the model then writes a reason, quotes the triggering provision and drafts safer wording. Three combinations (termination + no-refund + penalty; restraint + unlimited liability; auto-renewal + termination) are reported when all their rules occur anywhere in the contract; the graph joins the clauses involved. With Flesch reading ease F [16], legalese terms L per 100 words, cross-references X over n clauses and mean sentence length S, the complexity score is

$$Q = 0.35(100 - F) + 0.30\min(100, 8L) + 0.20\min(100, 40X/n) + 0.15\,\text{clip}(3(S - 15), 0, 100) \tag{3}$$

with grades A below 20, B below 40, C below 60, D below 80 and F above. The weights and cut-offs were chosen by the authors and are not validated against human ratings. The four rules of high severity are restraint of trade, unilateral termination, unlimited liability and IP overreach; penalty, one-sided arbitration, automatic renewal and no-refund are of medium severity. A combination is matched on rule identifiers anywhere in the contract, not on proximity or on which party is bound, so it is a coarse warning. The explanation prompt asks the model for a two- to three-sentence reason that quotes the clause, the exact triggering phrase and a fairer rewrite that keeps the clause's legitimate purpose; the summary prompt returns four to six sentences and four to eight key obligations from the first 40 clauses and the flag list.

E. Implementation

Passwords are bcrypt-hashed; a JWT (HS256, 1,440 minutes) guards every contract route, and a contract of another user returns 404. Uploads are limited to .pdf and .docx, written to a temporary file and deleted after analysis. Each contract costs one Gemini embedding per clause, one text call per flag and one for the summary, with three retries; a Gemini failure returns HTTP 502 and stores nothing. The default text model resolved to a model with a free quota of 20 requests a day and timed out when this study began, so all screens were made with gemini-3.1-flash-lite set on the server process only.

F. Data Model and Interfaces

Four tables hold the data: users; contracts, with the grade, the four component scores, the summary, the key obligations and the combinations as JSON; clauses, with text, page, type,

confidence, method and embedding; and flags, with rule, severity, reason, provision and safer wording. Deleting a user cascades to all of them, and the clause graph is not stored but rebuilt from clauses and flags on each request. The nine endpoints are health, register, login, upload, list, report, clause detail, graph and the evaluation metrics; a missing token gives 403, a bad or expired token 401, a wrong file type 400 and an unknown or foreign contract 404. Cross-origin access is open to all origins, which suits a local demonstration but not deployment.

G. Evaluation Procedure

The unmodified evaluation script was run on cached embeddings, so classification numbers come from the shipped code. Extra checks add contract-disjoint scoring (centroids rebuilt without training spans from the test span's contract), four baselines on the same split, exact-duplicate counting (four test spans also occur in training; excluding them changes top-1 from 0.792 to 0.797), and the confidence analysis. Rules were scored per clause and per rule as true and false positives and false negatives. The corpus run applies the non-API pipeline to every CUAD contract treated as one page. Output screens were captured with a browser driven by script, on three uploads made through the interface.

IV. RESULTS

A. Test Conditions

Classification and rules were scored on the data of Section III-B on the machine of Section I-A. Embeddings were cached so that the unmodified evaluation script and extra checks used identical vectors. Latencies are medians of 30 calls to 127.0.0.1 while other jobs ran on the machine. Table II lists the results.

TABLE II
MEASURED RESULTS

Measure	Result
Classification: top-1 / top-3 / top-5 (240 spans)	0.792 / 0.979 / 0.983
Contract-disjoint top-1; chance	0.800; 0.05
Baselines, top-1: TF-IDF + LR; 5-NN; embedding + LR	0.675; 0.746; 0.800
Tie-break triggers; Unclassified labels (of 240)	0; 0
Rules, shipped 40 clauses: P / R / F1	1.000 / 0.941 / 0.970
Rules, probe set 24+16: P / R / F1	0.786 / 0.458 / 0.579
Rules on 5,032 unrelated CUAD spans: any rule fires	37 (0.74%)
Same contract: DOCX / PDF / PDF, spaces collapsed	8 / 6 / 8 rules of 8
510 CUAD contracts, no API: time per contract	56 ms
Contracts with fewer than 5 clauses; flagged clauses	84 of 510; 2.2%
Login (bcrypt) / report / clause detail, median	254.7 / 9.3 / 5.0 ms
Upload to report: fair 8 clauses / CUAD PDF / one-sided	15.5 / 54.6 / 113.6 s
Automated tests	68 passed, 5 xfailed

B. Clause Classification

The nearest-centroid classifier reaches 0.792 top-1 and 0.979 top-3 on 20 categories (Fig. 3a), far above chance (0.05) and above TF-IDF with logistic regression (0.675); logistic regression on the same embeddings scores 0.800, so the shipped rule is as good as a trained classifier. Rebuilding the centroids without any training span from the same contract gives 0.800, so the split does not inflate the score. Governing Law, Renewal Term and IP Ownership Assignment are perfect; Non-Compete and Cap On Liability score 0.50 recall. The main confusions are Uncapped to Capped Liability (4), Non-Compete to No-Solicit of Customers (4) and the two no-solicit classes with each other (3 and 3).

The weak categories are the ones whose wording overlaps: No-Solicit Of Customers has recall 0.58 but precision only 0.44 because Non-Compete and employee no-solicit spans fall into it, Exclusivity, Liquidated Damages and Uncapped Liability score 0.67 recall, and Cap On Liability, whose spans often mention warranties or damages, is sent to Warranty Duration three times and to Uncapped Liability three times. Among the simple methods, 1-NN (0.604) and TF-IDF centroids (0.621) are weakest, and 5-NN reaches 0.746; the gain from averaging 25 vectors per class is therefore real. Classification is deterministic given the cached vectors, so no seed study applies; the open uncertainty is the 240-span sample, in which one span is 0.4 points.

Two parts of the rule never act. Top-1 similarity lies between 0.657 and 0.849 (median 0.767), so the 0.60 tie-break condition and the 0.35 floor are never met (Fig. 4): 152 of 240 spans pass the margin test but none passes the 0.60 test. Every clause therefore receives one of the 20 labels, including clauses that belong to none. On whole clauses of the three sample contracts (19 clauses, our manual reading, indicative only) about half of the labels were plausible: a payment clause was labelled Liquidated Damages and an unlimited-liability clause Cap On Liability, because the 20 categories describe annotated spans, not whole clauses with headings.

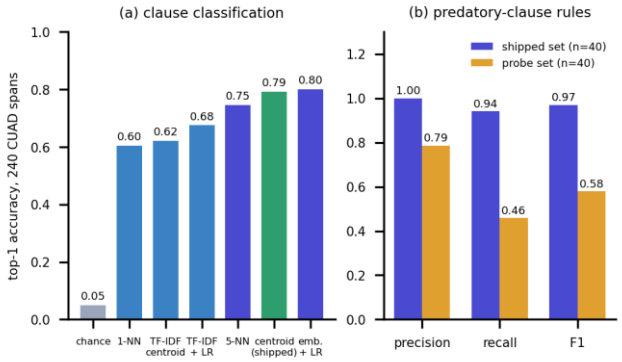


Fig. 3. (a) Top-1 accuracy of the shipped classifier and baselines. (b) Rule scores on the two test sets.

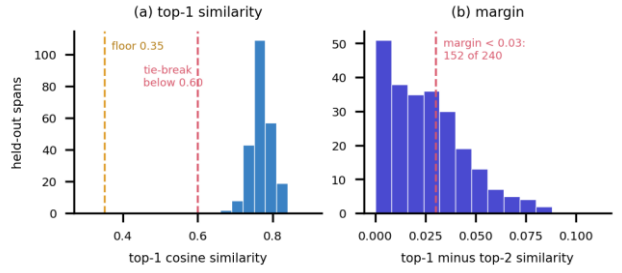


Fig. 4. Similarity of held-out spans to their nearest centroid and margin to the second.

C. Predatory-Clause Detection

On the 40 clauses shipped with the rules the detector scores precision 1.00, recall 0.94 and F1 0.97, with two misses. On the independent probe set the same rules reach

precision 0.79 and recall 0.46 (11 of 24 predatory clauses found, 3 false alarms; Fig. 3b). The misses are paraphrases without the keyword: "agrees not to accept any engagement from another software house", "liability shall be unlimited", "fees once paid cannot be returned" and "shall not be entitled to any refund" (the pattern expects "not entitled"). The false alarms are a fair, capped liquidated-damages clause, a conflict-of-interest bar flagged as restraint of trade, and mutual liability for gross negligence flagged as unlimited liability. The headline 97% F1 is therefore a measurement on the author's own phrasing; the Evaluation screen's wording "held-out" overstates it.

Per rule, recall on the probe set is 0.67 for auto-renewal, IP overreach and unilateral termination and 0.33 for the other five rules, with precision 0.5 for the penalty, restraint and unlimited-liability rules, each of which produced one false alarm. Because the probe set was written by us after reading the patterns, it is itself biased towards finding their weaknesses, and three examples per rule is small; the numbers show direction, not a precise rate. Still, the gap between 0.94 and 0.46 recall is too large to be sampling noise on clauses of this kind.

A cross-check on CUAD is only a proxy, since its categories are not predatory labels. The restraint rule fires on 21 of 257 Non-Compete spans (8.2%), the penalty rule on 68 of 121 Liquidated Damages spans (56.2%), the auto-renewal rule on 69 of 208 Renewal Term spans (33.2%), and the unlimited-liability and IP rules on none of 167 and 317 spans. On 5,032 spans of unrelated categories any rule fired 37 times (0.74%). Over all 510 contracts the non-API pipeline needs 56 ms per contract, flags 2.2% of 23,869 clauses, finds a risky combination in 11 contracts, and grades them A 4, B 169, C 268, D 68, F 1; 84 contracts give fewer than five clauses because their headings (for example "ARTICLE I") are not recognised.

D. Input Format and Output Screens

The same invented freelance contract gives all eight rules as DOCX but six as PDF: PDF text keeps visual line breaks, which the patterns' bounded gaps of the form .{0,N} cannot cross, and collapsing whitespace first restores all eight. Text before the first numbered heading is also dropped. Fig. 5 shows the report for the one-sided contract (grade B, score 23.3: wording, not fairness, is graded; 8 of 11 clauses flagged; three combinations), and Fig. 6 a flagged clause with its reason, quoted provision and safer wording. The fair contract gives no flag and grade A (13.7); the real CUAD supply agreement grade C (53.8) with one flag. Upload to report took 15.5 s, 54.6 s and 113.6 s, about 12 s per Gemini text call; the front end gives up at 180 s, so a contract with many flags would fail in the browser although the server would finish.

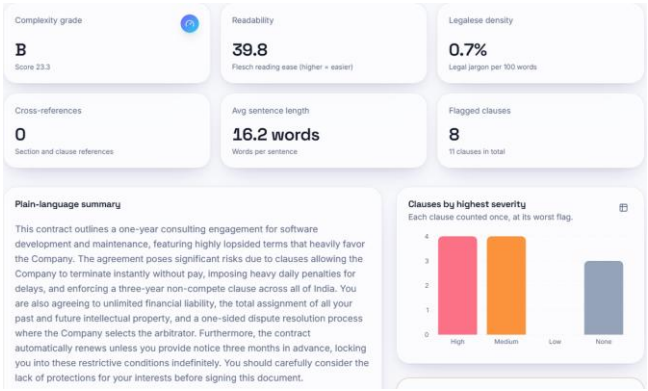


Fig. 5. Report screen for the one-sided freelance contract.

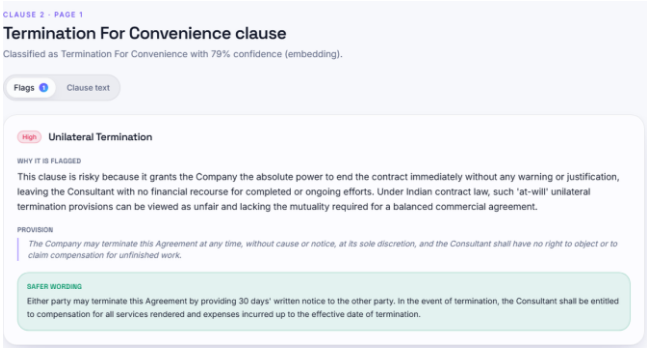


Fig. 6. Clause detail: reason, provision and safer wording.

The complexity score does not track fairness. The one-sided contract is graded B (23.3), the fair one A (13.7), and the real supply agreement C (53.8) with 43 cross-references. Across the 510 CUAD contracts the median Flesch score is 25.7, legalese density 1.13 per 100 words, cross-references 26 and mean sentence length 28.9 words; the score correlates with contract length at 0.30. The summaries were fluent but were not checked by a lawyer; for example the summary of the freelance contract says the assignment covers all of the signer's past and future intellectual property, a paraphrase that goes slightly beyond the clause text.

E. Testing and Cost

Sixty-eight tests pass and five are expected failures that document known limits (three keyword-free paraphrases, one fair capped liquidated-damages clause and malformed Gemini JSON answered with HTTP 400). They stub Gemini with deterministic vectors and JSON and run every API test on a temporary database. They cover each rule on a textbook clause and on benign clauses, the shipped-set score, the three combinations, the eight grade boundaries, segmentation, DOCX and PDF reading, the tie-break and its failure fallback, registration and login, missing, forged, expired and deleted-user tokens, the full upload, cross-user access, wrong file types, a missing key (503) and an upstream failure (502, with the temporary file removed). There are no browser tests, no load tests and no assertions on the quality of generated text. Local endpoints answer in 2 to 15 ms except bcrypt login and registration at about 255 to 265 ms; all measurable cost is in Gemini calls (about 12 s per text call here) and in quota, which on the default model allowed 20 text requests a day.

F. Threats to Validity

First, the shipped test set was written with the rules, so its scores are optimistic; the probe set was written by us and is small. Second, CUAD contains US filings, not Indian agreements, and its annotations are spans; no Indian corpus and no lawyer labels were available, so no claim is made about Indian practice beyond the statutes named in the rules. Third, the manual reading of 19 whole clauses is one reader's judgement. Fourth, latencies were measured on a shared machine and with a remote service whose speed changed during the study. Fifth, Gemini output is non-deterministic and was not repeated.

G. Comparison with the Base Paper

The base paper [1] reports 98.67% correct answers on 150 legal questions and a qualitative review of 50 contract drafts under Saudi law. ClauseGuard classifies clauses and flags predatory terms in Indian-law-oriented contracts: the datasets, tasks and metrics differ, so no head-to-head comparison is valid and none is claimed. We share its Gemini-based, jurisdiction-aware design, but we do not retrieve from a legal corpus, we add combination logic and a complexity grade, and we report negative results.

V. CONCLUSION

ClauseGuard works end to end: it reads contracts, classifies clauses at 0.792 top-1 on CUAD spans, flags eight Indian-law patterns, reports three risky combinations and an A-F grade, and explains each flag in plain language. The key finding is that the encouraging 0.97 F1 does not survive independent phrasing (0.58), and that PDF input and unreachable thresholds weaken the system further. Other limits are that no Indian contract corpus or lawyer review was available, the explanations are unverified, and free-tier quotas and 12 s calls constrain use.

Taking the objectives in turn: segmenting and classifying clauses is met, with a heading-based segmenter that fails on some styles and a classifier at 0.792 top-1; the RAG pipeline with the Claude API and pgvector is not met as stated, since Gemini and SQLite are used and nothing is retrieved from a legal corpus; the Neo4j knowledge graph is met only in part, by a per-contract graph of three combination patterns without legal concepts or precedents; predatory flags and combinations under Indian criteria are met by eight rules and three combinations of limited recall; the complexity score and plain-language summary are met, with unverified summaries; and the separate evaluation of classification and predatory detection is met and extended.

Future work: widen the rules with a legal-text classifier trained on labelled predatory clauses and score it on a held-out Indian set; normalise PDF text and recognise more heading styles; set the thresholds from the new embeddings and add an "unknown" class; retrieve statute text for the explanations and stream partial results.

REFERENCES

[1] F. Almutairy, S. Almutairi, M. Abdullah, and S. Salman, "SmartLaw Advisor: AI-powered legal consultation and contract analysis for Saudi Arabia," IEEE Access, vol. 14, pp. 32638 ff., 2026, doi: 10.1109/ACCESS.2026.3665420.

[2] A. A. Hagberg, D. A. Schult, and P. J. Swart, "Exploring network structure, dynamics, and function using NetworkX," in Proc. 7th Python in Science Conf. (SciPy), 2008, pp. 11-15.

[3] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force, May 2015.

[4] N. Provos and D. Mazieres, "A future-adaptable password scheme," in Proc. USENIX Annual Technical Conf. (FREENIX Track), 1999, pp. 81-91.

[5] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Machine Learning Research, vol. 12, pp. 2825-2830, 2011.

[6] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020, pp. 9459-9474.

[7] S. Liu, Z. Li, R. Ma, H. Zhao, and M. Du, "ContractEval: Benchmarking LLMs for clause-level legal risk identification in commercial contracts," arXiv:2508.03080, Aug. 2025.

[8] D. Hendrycks, C. Burns, A. Chen, and S. Ball, "CUAD: An expert-annotated NLP dataset for legal contract review," in Proc. NeurIPS Datasets and Benchmarks Track, 2021, arXiv:2103.06268.

[9] N. Lincoln, N. Whitehouse, J. Mar, and R. Perera, "A few good clauses: Comparing LLMs vs domain-trained small language models on structured contract extraction," arXiv:2605.05532, May 2026.

[10] D. Charan R, S. Siddarth, and M. Jeevitha, "Automated legal clause extraction and risk scoring using NLP and generative AI," Int. J. Res. Appl. Sci. Eng. Technol. (IJRASET), vol. 14, no. IV, Apr. 2026, doi: 10.22214/ijraset.2026.79369.

[11] I. Chalkidis, M. Fergadiotis, P. Malakasiotis, N. Aletras, and I. Androutsopoulos, "LEGAL-BERT: The muppets straight out of law school," in Findings of the Assoc. Computational Linguistics: EMNLP 2020, 2020, pp. 2898-2904.

[12] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in Proc. NAACL-HLT, 2019, pp. 4171-4186.

[13] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in Proc. EMNLP-IJCNLP, 2019, pp. 3982-3992.

[14] J. Lee et al., "Gemini Embedding: Generalizable embeddings from Gemini," arXiv:2503.07891, 2025.

[15] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," Information Processing and Management, vol. 24, no. 5, pp. 513-523, 1988.

[16] R. Flesch, "A new readability yardstick," J. Applied Psychology, vol. 32, no. 3, pp. 221-233, 1948.

[17] Government of India, "The Indian Contract Act, 1872 (Act No. 9 of 1872)," sections 27, 73 and 74.

[18] Government of India, "The Arbitration and Conciliation Act, 1996 (Act No. 26 of 1996)."

[19] Government of India, "The Consumer Protection Act, 2019 (Act No. 35 of 2019)."

[20] Government of India, "The Copyright Act, 1957 (Act No. 14 of 1957)."